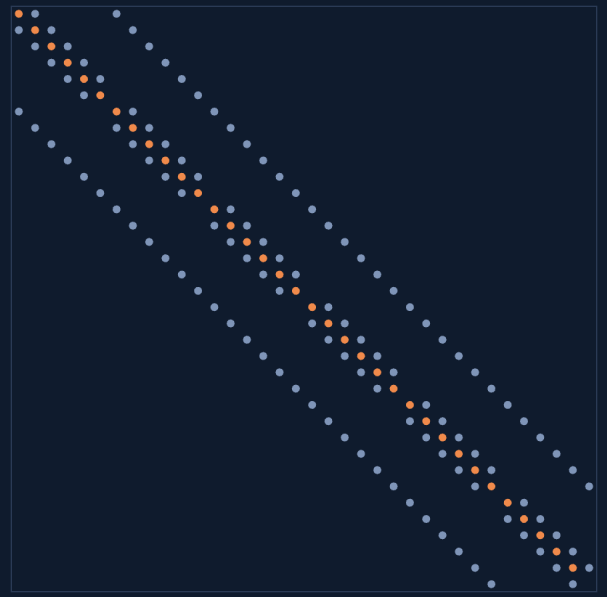
PARALELNI RAČUNARSKI SISTEMI · PROJEKAT P08

Paralelni solver rijetkih linearnih sistema

Kako brzo riješiti Ax = b sa milionima nepoznatih: metoda konjugovanih gradijenata, OpenMP, MPI i CUDA



Rijetka matrica 36 × 36 (2D Poissonov problem): ne-nule su samo na 5 dijagonala

Tim 1 · Tarik Fetahović, Hamza Husović, Tarik Lolić, Muhammed Sait Kaya

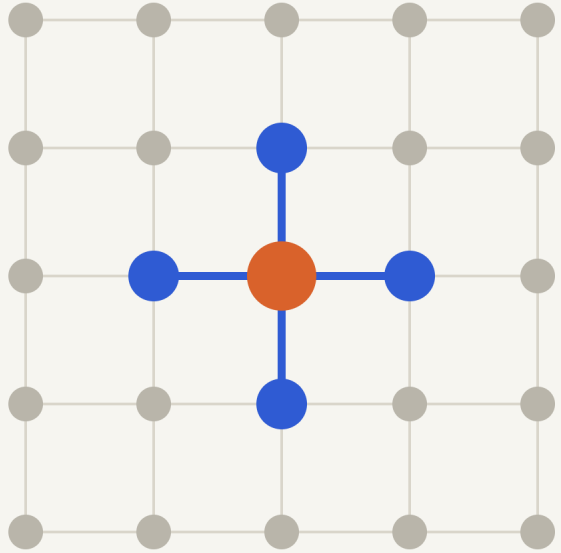
2. 11. 2026.

Svaka simulacija završava u Ax = b

1. Ploču podijelimo na mrežu tačaka i tražimo temperaturu u svakoj tački.
2. Temperatura tačke je prosjek njena 4 susjeda: **jedna jednačina po tački**.
3. Milion tačaka daje **milion jednačina sa milion nepoznatih**.

```
4·T13 - T8 - T12 - T14 - T18 = 0
```

Isto važi za čvrstoću konstrukcija, strujanje fluida, električna kola i naftna ležišta.



Narandžasto: tačka 13 · plavo: njena 4 susjeda (8, 12, 14, 18)

Skoro sve u toj matrici je nula

10^6

nepoznatih, dakle matrica od milion redova i milion kolona

7

brojeva po redu nije nula (u 3D: tačka i njenih 6 susjeda)

0,0007 %

matrice nosi informaciju; sve ostalo su nule

Čuvamo sve brojeve

8 TB

10^{12} polja × 8 bajtova: ne staje ni u jedan računar.

Čuvamo samo ne-nule (format CSR)

≈ 88 MB

7 miliona vrijednosti i njihovi indeksi: staje u svaki laptop, skoro 100 000 puta manje.

CSR: pamtimo samo ne-nule i gdje su

10	0	2	0
3	9	0	0
0	7	8	7
0	0	0	5

Matrica 4 × 4: 8 od 16 brojeva nije nula

val · vrijednosti, red po red

10 2 · 3 9 · 7 8 7 · 5

col · u kojoj koloni je svaka vrijednost

0 2 · 0 1 · 1 2 3 · 3

row_ptr · gdje u val počinje svaki red

0 2 4 7 8

```
for (i = 0; i < n; i++) {
  s = 0;
  for (k = row_ptr[i]; k < row_ptr[i+1]; k++)
    s += val[k] * x[col[k]];
  y[i] = s;    // SpMV: y = A·x
}
```

Direktno ili postepenim popravljanjem?

DIREKTNE METODE

Gaussova eliminacija, LU, Cholesky

+ Tačno, u poznatom broju koraka

– Popunjavanje: nule postaju brojevi

– Teško paralelno: korak čeka korak

Kod nas: samo za provjeru tačnosti na malim matricama

ITERATIVNE METODE · NAŠ IZBOR

CG, BiCGSTAB, GMRES

+ Matrica se samo množi vektorom

+ Memorija ostaje ≈ broj ne-nula

+ Svaki korak je lako paralelizovati

– Broj koraka zavisi od matrice

Ideja iterativnog pristupa: kreni od $x = 0$, izmjeri koliko promašuješ (rezidual $r = b - Ax$), popravi x i ponavljaj dok $\|r\| / \|b\|$ ne padne ispod 10^{-8}.

CG: svaka iteracija su tri operacije

Metoda konjugovanih gradijenata se „spušta niz dolinu“, a svaki novi pravac ne kvari napredak prethodnih. Za n nepoznatih stiže najkasnije za n koraka, u praksi mnogo ranije.

1 · SPMV, JEDNOM

$$q = A \cdot p$$

Čita cijelu matricu: najskuplji dio iteracije.

Paralelno: redovi su nezavisni

2 · SKALARNI PROIZVOD, 2×

$$\alpha = \rho \,/\, (p \cdot q)$$

Jedan broj iz svih elemenata: koliko daleko ići u pravcu p.

Globalni zbir: svi moraju sačekati

3 · AXPY, 3×

$$x \mathrel{+}= \alpha \cdot p$$

Dodaj vektor pomnožen brojem: pomjeri x i ažuriraj r.

Potpuno lokalno, bez čekanja

Plus prekondicioner $z = M^{-1} \cdot r$: jedna dodatna operacija po iteraciji, koja smanjuje broj iteracija. Ubrzamo li ove tri operacije, ubrzali smo cijeli solver.

Prekondicioner skraćuje put

Broj iteracija raste s izduženošću „doline“ (uslovljenost κ), otprilike kao $\sqrt{\kappa}$. Prekondicioner $M \approx A$, koji se lako rješava, čini dolinu okruglijom.

Prekondicioner	Manje iteracija	Paralelnost	Komunikacija
Jacobi (dijagonala)	malo	savršena	nema
Block-Jacobi + ILU(0)	srednje	dobra	nema
ILU(0) + level scheduling	dosta	ograničena	složena
AMG (samo za poređenje)	najviše	dobra	složena

Block-Jacobi slabi s brojem blokova; ILU rješava red po red, pa paralelizam postoji samo unutar „nivoa“. **Zato mjerimo ukupno vrijeme do rješenja, ne broj iteracija.**

Tri nivoa paralelizacije

OpenMP

Jedan računar; niti dijele memoriju

- Redovi podijeljeni po broju ne-nula
- reduction za skalarne proizvode
- Cijeli CG u jednom paralelnom regionu

MPI + OpenMP

Više procesa i računara; razmjenjuju poruke

- Svaki proces dobija blok redova
- Razmjena graničnih (halo) elemenata x
- MPI_Allreduce za globalne zbirove
- Računanje se preklapa s porukama

CUDA (opciono)

Hiljade niti grafičke kartice

- SpMV u formatu SELL-32-σ
- Svi vektori ostaju na GPU-u
- Poređenje s bibliotekom cuSPARSE

Isti kod CG-a za sve tri verzije: mijenja se samo implementacija SpMV-a, skalarnog proizvoda i AXPY-ja.

KLJUČNI UVID

Usko grlo je memorija, ne procesor

2

računske operacije po ne-nuli u SpMV-u (množenje i sabiranje)

≥ 12 B

pročitano iz memorije po ne-nuli (8 B vrijednost + 4 B indeks kolone)

≈ 0,17

operacija po bajtu: procesor najviše vremena čeka podatke

Zato se ubrzanje na jednom računaru izravnava nakon 4–8 jezgri, i to je očekivano. Kvalitet mjerimo u GB/s, u odnosu na stvarnu propusnost mašine (test STREAM).

Kako prilazimo tom plafonu

Balans po ne-nulama

Svaka nit dobija ≈ nnz/p elemenata, ne samo isti broj redova.

First-touch i vezivanje niti

Podaci leže uz procesorski čip niti koja ih koristi (NUMA).

Fuzija petlji

AXPY i skalarni proizvodi u jednom prolazu kroz memoriju.

Jedan paralelni region

Niti se ne pokreću ponovo za svaku operaciju.

Format SELL-C-σ

C redova odjednom, jednom vektorskom (SIMD) instrukcijom.

RCM preuređivanje

Ne-nule bliže dijagonali, pa vektor x ostaje u kešu.

Preklapanje u MPI-ju

Unutrašnji redovi se računaju dok poruke putuju.

Pipelined CG

Jedan globalni zbir po iteraciji, sakriven iza SpMV-a.

Prvi red: osnova OpenMP verzije, obavezna. Drugi red: nadogradnje formata i MPI verzije, koje mjerimo jednu po jednu.

Plan rada u osam faza

Faza 0 · jezgro

Sekvencijalna osnova

CSR, čitač .mtx, Poissonov generator, CG i testovi tačnosti.

Faza 1 · jezgro

OpenMP verzija

Paralelni kerneli, fuzija, first-touch, prvo mjerenje.

Faza 2 · jezgro

Prekondicioneri

Jacobi, block-Jacobi + ILU(0), level scheduling.

Faza 3 · jezgro

MPI + OpenMP

Podjela redova, halo razmjena, Allreduce.

Faza 4 · nadogradnja

SELL-C-σ i RCM

Vektorski SpMV i preuređivanje matrice.

Faza 5 · nadogradnja

Dodatni solveri

BiCGSTAB, pipelined CG, opciono GMRES.

Faza 6 · opciono

CUDA verzija

SpMV i cijeli CG na grafičkoj kartici.

Faza 7 · obavezno

Mjerenja i izvještaj

Skripte, grafici, poređenje s bibliotekom.

Kako mjerimo uspjeh

Tačnost

`‖b - Ax‖ / ‖b‖ < 10⁻⁸`

Nezavisna provjera reziduala i poređenje sa SciPy rješenjem.

Ubrzanje

`S(p) = T(1) / T(p)`

Strong i weak scaling, 1–N niti i procesa, uz efikasnost.

Propusnost

`GB/s naspram STREAM`

Cilj: 70–90 % izmjerene propusnosti mašine.

Poređenje

`Eigen ili PETSc`

Iste matrice, ista tolerancija, ukupno vrijeme do rješenja.

Test matrica	Redovi	Ne-nule	Zašto je u testu
Poissonova 3D (generisana)	do 27 000 000	≈ 7 po redu	weak scaling, pravilna
ldoor (SuiteSparse)	952 203	42 493 817	velika realna SPD matrica
FEM_3D_thermal2 (SuiteSparse)	147 900	3 489 300	test za BiCGSTAB
dc2 (SuiteSparse)	116 835	766 396	neujednačeni redovi

Istraživačka pitanja

PITANJE 1

Koliko paralelizacija ubrzava ili usporava rad za male, srednje i velike matrice?

Očekujemo (hipoteza): male matrice staju u keš, pa trošak sinhronizacije niti može nadmašiti posao i dati usporenje; srednje (u L3 kešu) daju najveće ubrzanje, moguće i superlinearno; za velike (u RAM-u) ubrzanje raste do zasićenja memorije, pa se izravna.

Mjerimo: Poissonova 3D, N = 10 do 250 (10^3 do $1{,}6 \cdot 10^7$ nepoznatih), 1–N niti, vrijeme po iteraciji.

PITANJE 2

Kakav je uticaj paralelnog izvršavanja u poređenju s jednom procesorskom jezgrom?

Očekujemo (hipoteza): isti broj iteracija i ista tačnost, a kraće vrijeme po iteraciji. Jedna jezgra koristi samo dio propusnosti memorije, pa je dobitak ograničen omjerom propusnosti cijelog čipa i jedne jezgre.

Mjerimo: $S(p) = T(1) / T(p)$ i efikasnost $E(p)$ za svaki kernel i cijeli CG, uz GB/s naspram STREAM-a.

UKRATKO

1. Rješavamo Ax = b sa milionima nepoznatih, u matrici u kojoj je skoro sve nula.
2. CG s prekondicionerom: svaka iteracija je SpMV, skalarni proizvodi i AXPY.
3. Paralelizujemo kroz OpenMP, MPI i CUDA; plafon je memorijska propusnost, i baš to mjerimo.

Hvala. Pitanja?

Tim 1 · Tarik Fetahović, Hamza Husović, Tarik Lolić, Muhammed Sait Kaya